

TREE 2

PROBLEM 2

QUESTION:

DS	Session	Tree 2	Question Information	Level 1 Challenge 72
Problem Description You are given an N-dimensional array A. You can execute an operation that removes the greatest and smallest elements from the array and replaces them with their difference. As a result, the array's size will drop by one after each operation. You are assigned Q jobs, each of which contains an integer K. After K operations, you must provide the total of all the items in the array for each task. Constraints: $2 \leq N \leq 10^5$ $1 \leq Q \leq 10^5$ $0 \leq A[i] \leq 10^9$ $0 \leq K < N$ Input: First line contains two space-separated integers N and Q, denoting the number of elements in array and number of queries respectively. Next line contains N space-separated integers denoting elements of the array. Next Q lines contain a single integer K. Output: For each task, print answer in a new line.				

Logical Test Cases

Test Case 1
INPUT (STDIN)
5 2
3 2 1 5 4
1
2
EXPECTED OUTPUT
13
9

Test Case 2
INPUT (STDIN)
5 3
13 12 11 15 14
1
2
3
EXPECTED OUTPUT
43
35
15

Mandatory Test Cases

Test Case 1
KEYWORD
for(i=0;i<n;i++)

Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
4

Test Case 2
TOKEN COUNT
300

Test Case 3
NLOC
38

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

int sort_ascending(const void* a, const void* b) {
    long long diff = *(const long long*)a - *(const long long*)b;
    return (diff > 0) ? 1 : ((diff < 0) ? -1 : 0);
}

int main() {
    int count, num_queries;
    if (scanf("%d %d", &count, &num_queries) != 2) return 0;

    long long values[count];
    long long initial_sum = 0;
    for (int i = 0; i < count; i++) {
        scanf("%lld", &values[i]);
        initial_sum += values[i];
    }

    long long prefix_results[count + 1];
    prefix_results[0] = initial_sum;

    int active_size = count;
    long long running_sum = initial_sum;
    for (int step = 1; step < count; step++) {
        qsort(values, active_size, sizeof(long long), sort_ascending);
        long long lowest = values[0];
        long long highest = values[active_size - 1];
        running_sum -= (2 * lowest);
        prefix_results[step] = running_sum;
        values[0] = highest - lowest;
        active_size--;
    }

    for (int q = 0; q < num_queries; q++) {
        int k;
        scanf("%d", &k);
        if (k < count) {
            printf("%lld\n", prefix_results[k]);
        } else {
            printf("%lld\n", prefix_results[count - 1]);
        }
    }
    return 0;
}
```

OUTPUT:

```
5 2
3 2 1 5 4
1
2
13
9
```

```
5 3
13 12 11 15 14
1
2
3
43
35
15
```